

UNION: 找不重覆結果，連結兩個SQL語法，
SELECT Date FROM Store_Information UNION SELECT Date FROM Internet_Sales
Minus: 遮罩，差集；A-B，A的值把B的值遮起來
子查詢: SELECT SUM(Sales) FROM Store_Information WHERE Store_name IN (SELECT store_name FROM Geography WHERE region_name = 'West')
EXISTS: SELECT SUM(Sales) FROM Store_Information WHERE EXISTS (SELECT * FROM Geography WHERE region_name = 'West')
CASE: 類似if/switch，
SELECT store_name, CASE store_name
WHEN 'Los Angeles' THEN Sales * 2
WHEN 'San Diego' THEN Sales * 1.5
ELSE Sales END "New Sales", Date FROM Store_Information
非'Los Angeles'或'San Diego'則不變。最後結果有store_name、"New Sales"、Date
建表格:
CREATE TABLE "表格名"("欄位 1" "欄位 1 型態", "欄位 2" "欄位 2 型態", ..)
例:
CREATE TABLE customer (First_Name char(50), Last_Name char(50), Birth_Date date)
CREATE TABLE Customer (SID integer CHECK (SID > 0))，"SID" 欄位大於 0 的整數
人造欄位: 流水欄位，與原本資料沒有關係的欄位
表格: 一般、暫存、虛擬
外來鍵: Customer_SID參照另一表單CUSTOMER裡的SID欄位
CREATE TABLE ORDERS (Order_ID integer, Amount double, Primary Key (Order_ID), Foreign Key (Amount) references CUSTOMER(Amount))
視圖: 視觀表、虛擬表格，一張表格透過條件選擇後的值並不存在(不占用記憶體)；
大表拆成小表，小表合成大表。V_Customer自定義名稱，SELECT開始自行採用SQL語法，
CREATE VIEW V_Customer AS SELECT First_Name, Last_Name, Country FROM Customer
CREATE -> ALERT(修改):
ALTER TABLE ORDERS ADD Gender char(1)，添加一欄位；DROP，刪除Gender；
CHANGE，改變欄位名稱，ALTER table customer change Address Addr char(50)；
MODIFY，改變資料值，ALTER table customer modify Addr char(30)
刪除表格: DROP TABLE "表格名稱"
清除表格: TRUNCATE TABLE "表格名稱"，資料消失但表格存在。

正規化: 降低重複性，避免更新(新增、刪除、修改)異常
無損失分解: 把一張表拆成數張表，資料不遺落
複合主鍵: 學號、姓名、性別相依於學號，課程名稱相依於課程代碼，成績相依於學號+課碼
第一正規化: 欄位長度(col數量)每欄一個值，沒有完全重複資料，主鍵外的欄位都與主鍵相依
第二正規化: 完全相依主鍵(仍允許複合主鍵)，不能部分功能相依(只有一主鍵無部份相依)
1.新增異常，如學號為主鍵，無法先新增課程給學生選，因主鍵為空
2.修改異常，如要改某一科目資料，同選修的學生太多，可能誤修改
3.刪除異常，刪減某科目後，導致無人可再選該科目
-> 分割: 學生資料表(學號、姓名、性別)，成績資料表(學號、課碼、成績)，
課程資料表(課程名稱等相依於課碼的資料)
第三正規化: 遞移(間接)相依。老師編號相依課程代碼，老師姓名相依老師編號
課程資料表又拆分出老師資料表
第三點五正規化: Boyce-Codd Normal Form(BCNF)，其它欄位不可相依其他非主鍵欄位，
即成績相依複合主鍵，且學號、課程代碼無相依非主鍵的成績

訂單序號 訂日 送日 產品代號 產品名稱 單價
客戶代號 客戶名稱 客戶地址
產品代號 訂購數量 產品名稱 單價 庫存量

送貨日期在2002/12/15之前、台北市的Price資料)-->語法(Select.....):
SELECT Price FROM Product
WHERE Pno IN
(SELECT Pno FROM Order_Item WHERE Ono IN
(SELECT Ono FROM Orders WHERE Ddate < '2002-12-15' AND Cno IN
(SELECT Cno FROM Customer WHERE Address = "台北市")))

附加寫法
SELECT Price FROM Product p
JOIN Order_Item ot ON p.Pno = ot.Pno
JOIN Orders o ON ot.Ono = o.Ono
JOIN Customer c ON o.Cno = c.Cno
WHERE o.Ddate < '2002-12-15' AND c.Address = '台北市'

虛擬路徑: 例如捷徑
套件: Flask_Uploads、Flask WTF，有很多更好用的套件
Flask-Sijax: 前端後端交換技術，解決等待後端回應的問題。
使用JSON在瀏覽器和伺服器之間傳遞資料。
不會刷新頁面就可看到回傳結果